



SPI Slave Project

Project Name: 4-Wire SPI Slave Interface

Author: Bassel Mahmoud

Date: August 1, 2026

Contents

1	Introduction	1
2	SPI Overview	1
2.1	Transaction Format	1
2.2	Frame Summary	1
3	Project Specification	2
3.1	Timing Constraints	2
4	Top Level Architecture	2
4.1	Block Diagram	3
5	Module Interface	3
5.1	spi_slave.sv	3
5.2	spi_memory.sv	3
5.3	spi_types_pkg.sv	4
5.4	spi_tb.sv	4
6	Internal Operation	4
6.1	Header Capture	4
6.2	Write Operation	4
6.3	Read Operation	4
6.4	Burst Operation	4
6.5	Reset and Transaction Boundaries	4
6.6	Control Flow	5
7	Timing Behavior	5
7.1	Write Transaction	5
7.2	Read Transaction	5
7.3	Burst Timing	6
8	Testbench Coverage	6
8.1	Observed Results	7
9	Limitations	7
10	Conclusion	7

A	Signal Summary	7
B	Important Notes	7

1 Introduction

Serial Peripheral Interface (SPI) is a simple synchronous serial protocol used for communication between a master device and one or more slaves. This project implements a 4-wire SPI slave that accepts chip select, clock, and serial input from an external master and returns serial output data when a read transaction is requested.

The design provides access to an internal register map through a 15-bit address space. It supports single-byte and burst transactions, uses MSB-first bit ordering, and follows the timing constraints described in the provided project specification.

2 SPI Overview

The implemented SPI slave uses the standard four signals:

- **CSB**: active-low chip select
- **SCLK**: serial clock from the master
- **SDI**: serial data input from the master
- **SDO**: serial data output to the master

The master begins a transaction by pulling **CSB** low, waiting for the lead time, and then shifting out a 16-bit header followed by one or more 8-bit data bytes. The slave samples the header, determines the direction of transfer, and either stores incoming data or drives outgoing data.

2.1 Transaction Format

Each SPI transfer begins with a 16-bit header:

- Bit [15] selects direction: 0 = **write**, 1 = **read**
- Bits [14:0] hold the starting register address

After the header, the transaction continues with one or more data bytes. There is no dedicated burst bit. Burst mode is detected when the master keeps **CSB** low and continues clocking additional data bytes.

2.2 Frame Summary

Table 1: SPI Frame Fields

Field	Bits	Description
R/W	[15]	Selects write or read transaction
Address	[14:0]	Starting location in the register map
Data	8 per byte	Payload bytes, MSB first

3 Project Specification

The RTL was written to satisfy the following requirements:

- The slave must support both write and read transactions.
- The address space is 15 bits wide, giving 32768 byte locations.
- The register map must be readable combinatorially for read turnaround.
- Burst transfers must auto-increment the address after each byte.
- t_{lead} must exist between CSB assertion and the first SCLK edge.
- t_{lag} must exist between the final clock edge and CSB deassertion.

3.1 Timing Constraints

The specification requires a half-cycle turnaround on reads. After the last address bit is captured, the slave must prepare the first output bit for the very next serial clock edge. This is why the memory read path is combinational.

Design Assumption

The project assumes $t_{\text{lag}} = t_{\text{lead}}$. The testbench uses a fixed delay on both sides of the serial transfer to satisfy this requirement.

4 Top Level Architecture

The project has three main pieces:

- the SPI slave controller
- the internal memory/register map
- the testbench that drives transactions and checks results

4.1 Block Diagram

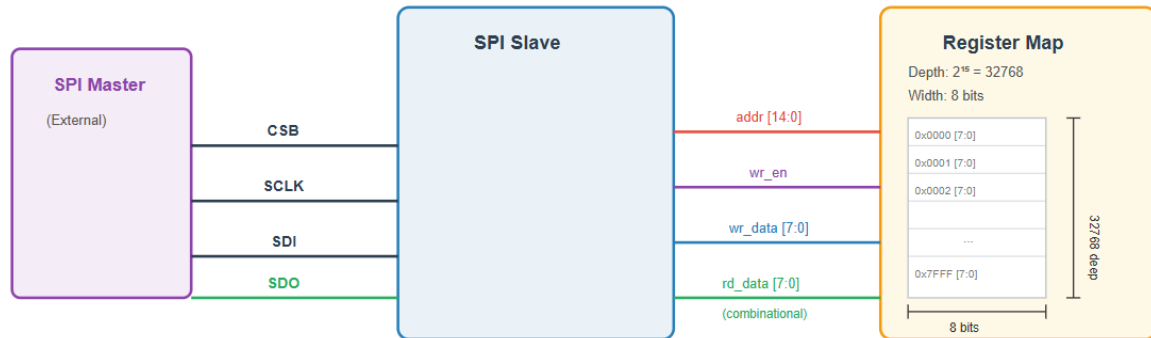


Figure 1: SPI Slave Top-Level Block Diagram

5 Module Interface

5.1 spi_slave.sv

This module implements the serial protocol engine. It receives `sclk`, `csb`, `sdi`, and `rst_n`, and produces `sdo`, `addr`, `wr_data`, and `wr_en`.

Table 2: SPI Slave Signals

Signal	Direction	Width	Description
<code>sclk</code>	In	1	Serial clock from master
<code>rst_n</code>	In	1	Active-low reset
<code>sdi</code>	In	1	Serial data input
<code>csb</code>	In	1	Active-low chip select
<code>rd_data</code>	In	8	Combinational read data from memory
<code>sdo</code>	Out	1	Serial data output
<code>addr</code>	Out	15	Register address
<code>wr_data</code>	Out	8	Write byte data
<code>wr_en</code>	Out	1	Write strobe

5.2 spi_memory.sv

This module implements the internal register map as a 32768-deep array of 8-bit values. The read port is combinational, and the write port is synchronous on the memory clock. In simulation, it initializes from `mem_init.txt`.

5.3 spi_types_pkg.sv

This package holds shared types for the design:

- `rw_e`: write/read enumeration
- `spi_header_str`: packed header containing `rw` and `addr`

5.4 spi_tb.sv

The testbench generates the SPI clock, controls `CSB`, drives serial data, and checks the response of the slave and memory model.

6 Internal Operation

6.1 Header Capture

The slave counts serial clock edges and captures the first 16 bits as the transaction header. Bit 15 defines the direction and bits 14 through 0 define the starting address.

6.2 Write Operation

For write transactions, data bits are shifted in MSB first. When the byte completes, the slave copies the assembled byte into `wr_data` and asserts `wr_en` so the memory can store the value.

6.3 Read Operation

For read transactions, the slave loads `rd_data` into a shift register and shifts the bits out on `SD0` MSB first. The output is tri-stated outside read data phases.

6.4 Burst Operation

If the master keeps `CSB` low after the first byte, the slave continues treating the transaction as a burst. The address is incremented after each byte so the next byte accesses the next register location.

6.5 Reset and Transaction Boundaries

When reset is asserted or `CSB` is high, the controller clears its internal counter and temporary registers. This ensures every new transaction begins from a known state.

6.6 Control Flow

The design uses a simple counter-based flow:

- counters count the header and data bits
- a small packed header stores direction and address
- a write shift register assembles outgoing memory writes
- a read shift register serializes memory output

7 Timing Behavior

7.1 Write Transaction

During a write, the slave samples the 16-bit header first and then captures the incoming byte(s). SDO remains high impedance throughout the write data phase.

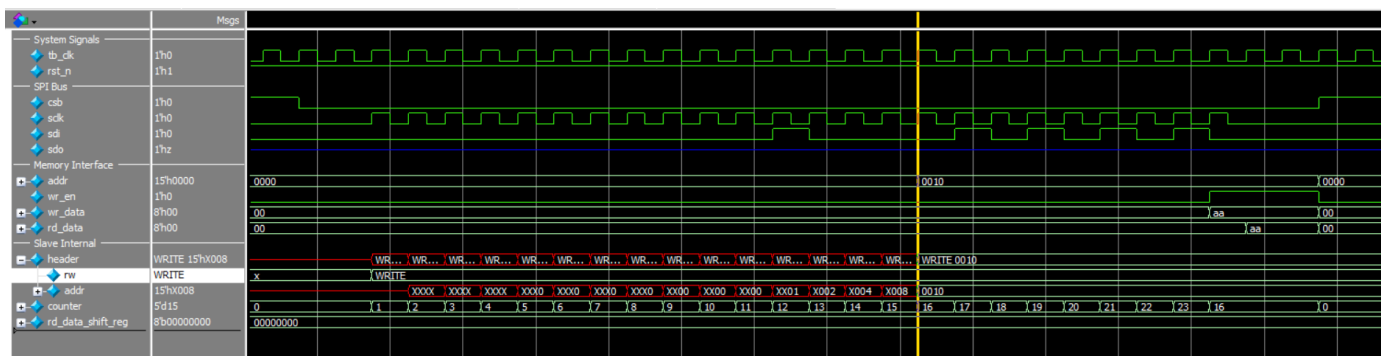


Figure 2: SPI Slave Write Transaction

7.2 Read Transaction

During a read, the slave captures the header and then immediately prepares to drive SDO. The memory output must be available quickly enough to satisfy the next serial clock edge.

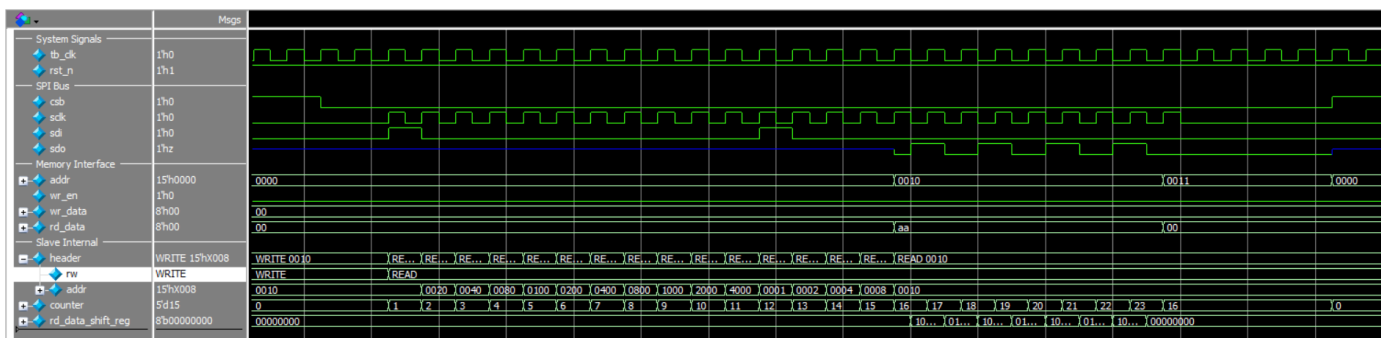


Figure 3: SPI Slave Read Transaction

7.3 Burst Timing

Burst timing is the same as single-byte timing, except the master continues to hold CSB low and keeps clocking additional bytes.

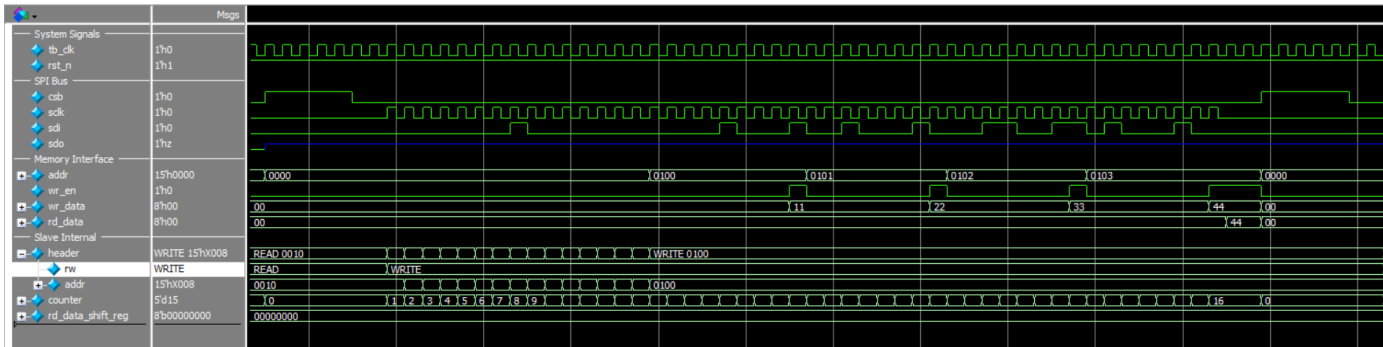


Figure 4: SPI Slave Burst Write Transaction

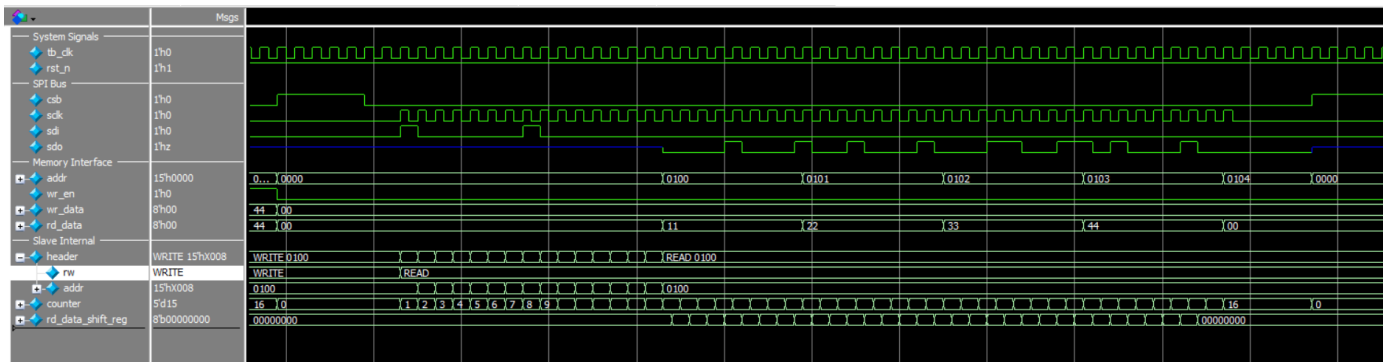


Figure 5: SPI Slave Burst Read Transaction

8 Testbench Coverage

The testbench covers:

1. single-byte write to address 0x0010
2. single-byte read from address 0x0010
3. burst write to address 0x0100
4. burst read from address 0x0100
5. burst write across a small address boundary
6. burst readback verification

These cases confirm that the slave stores and returns the expected bytes and that the address auto-increment behavior works correctly.

8.1 Observed Results

The testbench prints the read-back values after each read transaction. Based on the written stimulus, the expected values are preserved in the register map and returned in order.

9 Limitations

The current implementation intentionally keeps the feature set small. It does not include:

- extra SPI modes beyond the implemented 4-wire transfer
- protocol error reporting
- address wrapping logic beyond linear increment
- multi-master arbitration or shared-bus behavior

10 Conclusion

This project implements a practical SPI slave controller with a combinational readable register map, support for single and burst transactions, and timing behavior that matches the project requirements. The design is compact, easy to verify, and directly aligned with the provided specification.

A Signal Summary

Table 3: Top-Level Summary

Signal	Direction	Width	Description
csb	In	1	Active-low chip select
sclk	In	1	Serial clock
sdi	In	1	Serial input
sdo	Out	1	Serial output
addr	Out	15	Memory address
wr_data	Out	8	Write payload byte
wr_en	Out	1	Memory write enable
rd_data	In	8	Memory read data

B Important Notes

- The design is MSB-first for both the header and data bytes.
- Read data is taken directly from the combinational memory output.
- Burst behavior is inferred from continued clocking while **CSB** remains low.
- The register map depth is 32768 bytes, matching the 15-bit address field.